

同濟大學

TONGJI UNIVERSITY

信也科技 DGraph 金融反欺诈图算法挑战赛
综合实践

学 院	计算机科学与技术学院
专 业	数据挖掘
队 名	大展宏图鹏程万里明察秋毫建功立业
学生姓名	程亦诚, 胡捷鸣, 李盛鹏, 胡桥健, 张宏斌
学 号	2351582, 2351584, 2351136, 2351295, 2352027
指导教师	陈宇飞
日 期	2025 年 12 月 18 日

课题名称

摘要

随着互联网金融的快速发展，金融反欺诈已成为风控领域的重要挑战。本文基于信也科技 DGraph 金融反欺诈挑战赛的大规模动态图数据，研究了两种不同的技术路线来解决用户欺诈识别问题。第一种方法采用传统机器学习思路，通过精心设计的 198 维特征工程结合集成学习策略，在测试集上达到 0.83 的 AUC 值。第二种方法采用图神经网络技术，通过三层 GATv2 架构端到端学习节点表示，实现了 0.79 的 AUC 值。实验结果表明，基于特征工程的基线方法在当前任务上表现更优，但 GNN 方法展现了端到端学习的潜力。本文的工作为金融风控领域提供了有价值的技术方案和实践经验。

关键词：金融反欺诈、图神经网络、特征工程、集成学习、图注意力网络、类别不平衡处理

目 录

1	引言	1
2	数据集描述与特征工程	2
2.1	数据集概述	2
2.2	数据预处理	2
2.3	大规模特征工程	3
2.3.1	时间窗口特征	3
2.3.2	基础统计特征	4
2.3.3	高级结构特征	5
2.3.4	边类型多样性特征	5
3	方法一：基于特征工程与集成学习的基线方法	7
3.1	方法概述	7
3.2	特征融合策略	7
3.3	类别不平衡处理	7
3.4	多模型集成策略	8
3.5	模型训练与验证	8
3.6	AUC 加权融合	9
3.7	测试集预测与结果生成	10
3.8	性能分析	11
4	方法二：基于图注意力网络的深度学习方法	12
4.1	方法概述	12
4.2	整体架构	12
4.3	核心工作流程与特征工程	12
4.3.1	高级度数特征	12
4.3.2	多尺度时序特征	15
4.4	增强的 GAT 模型架构	15
4.4.1	残差连接机制	16
4.5	高级 Focal Loss	17
4.6	模型训练流程	17
4.6.1	动态学习率调度	17
4.7	性能分析与对比	19
4.7.1	实验结果	19
4.7.2	与传统方法对比	19
4.8	关键超参数配置	19

4.8.1	模型参数	19
4.8.2	优化器参数	20
4.8.3	损失函数参数	20
4.8.4	训练参数	20
4.9	工程优化与创新	20
4.9.1	大规模图处理策略	20
4.9.2	GPU 内存优化	20
4.10	方法总结	21
5	结论与展望	22
5.1	方法性能分析	22
5.2	技术创新点	22
5.3	实际应用价值	22
5.4	局限性与未来方向	23
5.5	结语	23

1 引言

随着互联网金融的快速发展，线上信贷业务面临日益隐蔽化、专业化、规模化的欺诈挑战。如何有效识别、预测和预防欺诈行为，已成为金融科技领域亟待解决的重要问题。图神经网络作为近年来迅速发展的关键技术，在识别复杂关联风险方面展现出独特优势。信也科技联合浙江大学公开的 DGraph 大规模动态图数据集，为研究金融反欺诈提供了宝贵的真实场景数据。

本次挑战赛的核心任务是基于大规模动态图数据的节点二分类任务，要求参赛者利用提供的图结构、时序信息和节点属性信息，构建高效的图神经网络模型，识别隐藏在海量用户中的欺诈个体。数据集包含四类节点：正常用户（Class 0）、欺诈用户（Class 1），以及两类背景用户（Class 2 和 Class 3）。预测任务为典型的二分类问题：仅需将前景节点（Class 0 和 Class 1）中的欺诈用户（Class 1）区分出来。

面对这个具有挑战性的任务，本文提出了两种不同的方法来解决。第一种是基于传统机器学习的基线方法，采用大规模特征工程结合集成学习策略，通过精心设计的 200 多维特征捕捉用户的各种统计和结构信息。第二种是基于图神经网络的深度学习方法，利用图注意力网络（GAT）直接学习节点表示，并配合高级的特征工程技术。

数据集具有以下显著特点：(1) 大规模动态图：包含 4,024,623 个节点和 4,927,620 条有向边；(2) 极端类别不平衡：欺诈用户占比约 1%；(3) 背景节点占比高：超过 60% 的节点为背景节点，虽不直接参与评估但对拓扑结构至关重要；(4) 时间信息丰富：每条边都带有时间戳，反映了用户关系的动态演化。

两种方法在测试集上都取得了优异的性能。基线方法通过融合 HistGradientBoosting、XGBoost 和 LightGBM 等多个模型，达到了 0.83 的 AUC 值；而图神经网络方法通过三层 GATv2 结构的增强模型，实现了 0.79 的 AUC 值。这些结果充分展示了不同技术路线在金融反欺诈任务中的有效性和应用价值。

本文后续章节将详细描述两种方法的特征工程、模型架构和训练策略，并对比分析它们的优缺点。通过系统性的实验和深入的分析，我们希望能够为金融风控领域的研究和实践提供有价值的参考和启示。

2 数据集描述与特征工程

2.1 数据集概述

本次实验使用的数据集来源于信也科技真实信贷风控反欺诈场景，经过严格脱敏处理。数据集存储在 `phase1_gdata.npz` 文件中，包含大规模动态图信息。通过 Python 加载和分析，我们发现数据集的具体情况如下：

```
# 加载数据集
data = np.load("phase1_gdata.npz", allow_pickle=True)

# 提取数据
x = data["x"].astype(np.float32)          # 节点特征 (N_node, 17)
y = data["y"].squeeze()                   # 节点标签 (N_node,)
edge_index = data["edge_index"].astype(np.int64)    # 有向边 (N_edge, 2)
edge_type = data["edge_type"].squeeze()    # 边类型 (N_edge,)
edge_timestamp = data["edge_timestamp"].squeeze()  # 边时间戳 (N_edge,)
train_mask = data["train_mask"].astype(np.int64)   # 训练集索引
test_mask = data["test_mask"].astype(np.int64)     # 测试集索引

N = x.shape[0]    # 4,024,623个节点
E = edge_index.shape[0]    # 4,927,620条边
```

数据集具有以下显著特征：(1) 超大规模图结构：包含 402 万多个用户节点和 492 万多条有向边；(2) 丰富的节点特征：每个节点具有 17 维初始特征，可能包含用户的基本属性、行为统计等信息；(3) 动态时间信息：每条边都带有时间戳，表示关系建立的时间；(4) 多样化的边类型：边类型用整数编码，表示不同类型的用户关系；(5) 极端类别不平衡：欺诈用户（Class 1）仅占约 1%，正常用户（Class 0）占绝大多数。

数据集中的节点分为四类：Class 0（正常用户）、Class 1（欺诈用户）、Class 2 和 Class 3（背景用户）。背景节点虽不直接参与评估，但其构成的网络拓扑信息对于挖掘潜在欺诈关系至关重要。这种设定增加了任务的挑战性，要求模型能够有效利用无标签节点的结构信息。

2.2 数据预处理

在进行特征工程之前，需要对原始数据进行预处理。首先处理标签数据，将多分类问题转化为二分类问题：

```
# 将多分类标签转换为二分类
```

```
y_bin = np.zeros(N, dtype=np.int64)
mask_label = (y != -100) # 去除测试集标签
y_bin[mask_label] = (y[mask_label] == 1).astype(np.int64)
```

```
# 划分训练集和验证集
train_idx = train_mask[y[train_mask] != -100]
train_idx_local, val_idx_local = train_test_split(
    train_idx,
    test_size=0.1,
    random_state=42,
    stratify=y_bin[train_idx]
)
```

预处理还包括度数的基本计算，这是后续特征工程的基础：

```
# 计算基本的度数特征
out_deg = np.bincount(edge_index[:, 0], minlength=N).astype(np.float32)
in_deg = np.bincount(edge_index[:, 1], minlength=N).astype(np.float32)
deg = in_deg + out_deg
deg_diff = out_deg - in_deg
```

2.3 大规模特征工程

针对数据集的特点，我们设计了包含 200 多个维度的综合特征体系，从多个角度刻画用户的特征。

2.3.1 时间窗口特征

时间窗口特征捕捉用户在不同时间尺度下的活跃程度。我们定义了多个时间窗口来统计用户的近期活跃情况：

```
# 定义时间窗口
max_day = int(edge_timestamp.max())
win_base = np.array([3, 7, 14, 30, 60, 90, 180], dtype=np.int32)
win_days = np.concatenate([win_base, max_day - win_base])
win_threshold = max_day - win_days
```

```
# 计算每个窗口内的活跃度
```

```
edge_ts = edge_timestamp.reshape(-1, 1)
mask = edge_ts >= win_threshold.reshape(1, -1)

# 统计每个节点在各时间窗口的活跃度
nodes_flat = np.concatenate([edge_index[:, 0], edge_index[:, 1]])
mask_flat = np.concatenate([mask, mask], axis=0)
recent_feats = np.zeros((N, W), dtype=np.float32)
for w in range(W):
    recent_feats[:, w] = np.bincount(
        nodes_flat,
        weights=mask_flat[:, w].astype(np.float32),
        minlength=N
    )
```

这种多尺度时间窗口设计能够捕捉用户的短期、中期和长期行为模式，对于识别异常行为模式具有重要意义。

2.3.2 基础统计特征

基础统计特征包括度数相关特征和时间相关特征：

```
# 时间统计特征
min_day = np.full(N, 1e9, dtype=np.float32)
max_day_node = np.full(N, -1e9, dtype=np.float32)
ts_flat = np.concatenate([edge_timestamp, edge_timestamp])

# 计算每个节点的最早和最晚活跃时间
np.minimum.at(min_day, nodes_flat, ts_flat)
np.maximum.at(max_day_node, nodes_flat, ts_flat)

# 计算活跃时间跨度
active_span = max_day_node - min_day
active_span[active_span < 0] = 0

# 时间加权度数
Tmax = max_day_node.max() + 1e-6
time_weight = ts_flat / Tmax
```



```
w_out = np.zeros(N, dtype=np.float32)
w_in = np.zeros(N, dtype=np.float32)
np.add.at(w_out, edge_index[:, 0], time_weight[:, E])
np.add.at(w_in, edge_index[:, 1], time_weight[:, E])
time_weighted_deg = w_out + w_in
```

这些特征反映了用户的活跃程度、行为持续性和时间规律性，是识别欺诈行为的重要依据。

2.3.3 高级结构特征

为了更深入地挖掘图结构信息，我们计算了二阶和三阶邻居特征：

```
# 构建邻接矩阵
rows = edge_index[:, 1] # 入邻居
cols = edge_index[:, 0] # 出邻居
data = np.ones(E, dtype=np.float32)
A_in = coo_matrix((data, (rows, cols)), shape=(N, N))
A_out = coo_matrix((data, (cols, rows)), shape=(N, N))

# 计算二阶邻居特征
A2_in = A_in.dot(A_in)
A2_out = A_out.dot(A_out)
num_2hop_in = np.array(A2_in.sum(axis=1)).flatten()
num_2hop_out = np.array(A2_out.sum(axis=1)).flatten()

# 计算三元组结构
triangles_in_out = np.array(A_in.dot(A_out).diagonal()).astype(np.float32)
triangles_out_in = np.array(A_out.dot(A_in).diagonal()).astype(np.float32)
```

高阶结构特征能够捕捉用户在社交网络中的局部团簇特性，对于识别欺诈团伙具有重要意义。

2.3.4 边类型多样性特征

边类型反映了用户关系的多样性，我们统计了每个节点的边类型分布：

```
# 统计边类型
num_types = int(edge_type.max() + 1)
in_type_count = np.zeros((N, num_types), dtype=np.float32)
out_type_count = np.zeros((N, num_types), dtype=np.float32)
```

```
np.add.at(out_type_count, (edge_index[:,0], edge_type), 1)
np.add.at(in_type_count, (edge_index[:,1], edge_type), 1)

# 计算类型比率
in_type_ratio = in_type_count / np.maximum(in_type_count.sum(axis=1, keepdims=True), 1)
out_type_ratio = out_type_count / np.maximum(out_type_count.sum(axis=1, keepdims=True), 1)

# 计算类型熵
from scipy.stats import entropy
type_entropy_in = entropy(in_type_ratio.T + 1e-6)
type_entropy_out = entropy(out_type_ratio.T + 1e-6)
```

边类型多样性特征可以识别用户的行为模式，正常用户通常具有相对稳定的关系类型分布，而欺诈用户可能表现出异常的类型模式。

通过上述特征工程，我们构建了一个包含 198 维特征的特征矩阵，为后续的模型训练提供了丰富的信息基础。

3 方法一：基于特征工程与集成学习的基线方法

3.1 方法概述

第一种方法采用传统机器学习的思路，通过大规模特征工程结合集成学习策略来解决金融反欺诈问题。该方法的核心思想是通过精心设计的手工特征来充分挖掘图结构信息，然后使用多个梯度提升树模型进行集成预测。这种方法的优势在于特征的可解释性强，模型训练效率高，并且在大规模数据集上表现出色。

3.2 特征融合策略

在完成 198 维特征工程后，我们将原始特征与工程特征进行融合，构建最终的训练特征矩阵：

```
# 融合原始特征和工程特征
X = np.concatenate([x, struct_feats], axis=1).astype(np.float32)
D = X.shape[1] # 最终特征维度：215
print("Final X:", X.shape)
```

特征融合的策略是将原始的 17 维节点特征与 198 维工程特征拼接，形成 215 维的最终特征表示。这种融合方式保留了原始信息的完整性，同时补充了丰富的统计和结构特征。

3.3 类别不平衡处理

面对极端类别不平衡的问题（正负比例约为 1:100），我们采用了数据分块训练的策略：

```
# 找到正负样本索引
pos_idx = np.where(y_train == 1)[0]
neg_idx = np.where(y_train == 0)[0]

# 将负样本分成多个块
chunk_size = int(len(pos_idx)) # 每块负样本数量等于正样本数
neg_chunks = [neg_idx[i:i+chunk_size] for i in range(0, len(neg_idx), chunk_size)]
neg_chunks = neg_chunks[:9] # 使用前9块

# 为每块生成训练索引
train_chunks_idx = []
n_pos_sample = int(len(pos_idx))
for neg_chunk in neg_chunks:
    pos_sample_idx = np.random.choice(pos_idx, size=n_pos_sample, replace=False)
```

```
train_idx_chunk = np.concatenate([pos_sample_idx, neg_chunk])
np.random.shuffle(train_idx_chunk)
train_chunks_idx.append(train_idx_chunk)
```

这种分块策略确保了每个训练子集中正负样本比例平衡 (1:1)，同时充分利用了大量的负样本信息。通过训练多个子模型并集成，可以有效缓解类别不平衡带来的影响。

3.4 多模型集成策略

我们采用了三种不同的梯度提升树模型进行集成，分别是 HistGradientBoosting、XGBoost 和 LightGBM：

```
# 定义模型列表
models = [
    {"name": "HistGB", "cls": HistGradientBoostingClassifier},
    {"name": "XGBoost", "cls": XGBClassifier},
    {"name": "LightGBM", "cls": LGBMClassifier},
]

# 分配训练块给不同模型
n_chunks = len(train_chunks_idx)
part_size = n_chunks // len(models)
model_chunks = [train_chunks_idx[i*part_size : (i+1)*part_size
                    if i<len(models)-1 else n_chunks]
                 for i in range(len(models))]
```

每个模型使用不同的训练块子集，这种策略增加了模型的多样性，有助于提升集成的效果。HistGradientBoosting 具有较好的鲁棒性，XGBoost 在处理特征交互方面表现优异，而 LightGBM 则以训练速度快著称。

3.5 模型训练与验证

对于每个子模型，我们使用相应的训练块进行训练，并在验证集上评估性能：

```
# 训练每个子模型
for model_idx, model_info in enumerate(models):
    name = model_info["name"]
    cls = model_info["cls"]
    chunks = model_chunks[model_idx]
```

```
for i, train_idx_chunk in enumerate(chunks):
    X_train_chunk = X_train[train_idx_chunk]
    y_train_chunk = y_train[train_idx_chunk]

    # 训练模型
    clf = cls() if callable(cls) else cls
    clf.fit(X_train_chunk, y_train_chunk)
    trained_models.append(clf)

    # 验证
    val_prob = clf.predict_proba(X_val)[: , 1]
    val_auc = roc_auc_score(y_val, val_prob)
    print(f"    >> {name} chunk {i+1}/{len(chunks)} AUC = {val_auc:.4f}")
```

训练过程中，我们监控每个子模型在验证集上的 AUC 值，以确保模型的有效性。所有子模型都使用默认参数，避免了复杂的超参数调优过程。

3.6 AUC 加权融合

为了获得最终的预测结果，我们采用 AUC 加权融合策略，即根据每个子模型在验证集上的 AUC 表现分配权重：

```
# 计算每个子模型的AUC作为权重
auc_list = []
for val_prob in val_probs_all:
    auc = roc_auc_score(y_val, val_prob)
    auc_list.append(auc)

# 归一化权重
auc_array = np.array(auc_list)
weights = auc_array / auc_array.sum()

# 加权融合预测结果
val_probs_all = np.array(val_probs_all)
ensemble_prob = np.average(val_probs_all, axis=0, weights=weights)
```

AUC 加权融合的原理是赋予表现更好的模型更大的权重，这种自适应的融合策略能够最大化集成的效果。相比简单平均，AUC 加权能够充分利用各模型的相对优势。

3.7 测试集预测与结果生成

在获得集成模型后，我们对测试集进行预测并生成提交文件：

```
# 对测试集进行预测
test_feats = X[test_mask]
test_probs_all = []

for clf in trained_models:
    # 预测概率
    if hasattr(clf, "predict_proba"):
        prob = clf.predict_proba(test_feats)[: , 1]
    else:
        prob = clf.decision_function(test_feats)
        prob = (prob - prob.min()) / (prob.max() - prob.min())

    test_probs_all.append(prob)

# AUC加权融合
test_probs_all = np.array(test_probs_all)
test_ensemble_prob = np.average(test_probs_all, axis=0, weights=weights)

# 生成提交格式
submission = np.vstack([
    1.0 - test_ensemble_prob,
    test_ensemble_prob
]).T.astype(np.float32)

np.save("submission.npy", submission)
```

提交文件的格式为 (N_test, 2)，每行包含预测为 Class 0 和 Class 1 的概率，且两列概率之和为 1。这种格式符合比赛要求，便于在线评估系统计算 AUC。

3.8 性能分析

基线方法在验证集上达到了 0.832 的 AUC 值，表现出色。该方法的优势包括：

(1) 强大的特征表达能力：通过精心设计的 198 维特征，全面捕捉了用户的多维度信息，包括时序行为、图结构位置、关系类型等。

(2) 有效的类别不平衡处理：分块训练策略确保每个子模型看到平衡的数据分布，同时通过集成充分利用了所有负样本信息。

(3) 鲁棒的集成策略：多模型融合结合 AUC 加权，充分发挥了不同模型的优势，提高了预测的稳定性和准确性。

(4) 计算效率高：整个过程在单机上完成，不需要特殊的硬件资源，适合实际生产环境部署。

该方法的局限性在于特征工程需要人工设计，依赖于领域知识和经验。同时，对于深层的高阶图结构信息，手工特征可能无法充分捕捉。尽管如此，该方法在测试集上 0.83 的 AUC 成绩证明了其在金融反欺诈任务中的有效性。

装
订
线

4 方法一：基于图注意力网络的深度学习方法

4.1 方法概述

本项目采用图神经网络的思路，通过深度学习模型直接学习节点的表示和分类。该方法使用增强的图注意力网络（GATv2）作为核心架构，配合 GPU 并行计算的特征工程和训练策略。GAT 方法能够端到端地学习图结构信息，自动发现对分类任务有用的特征模式，在测试集上达到了 **0.82898558164** 的 AUC 值。

选择 GAT 的主要原因包括：

- **图数据适配性**：金融交易数据天然构成图结构，GAT 能够同时利用节点特征和图结构信息
- **注意力机制优势**：自适应学习不同邻居节点的重要性权重，提供强可解释性
- **处理异质性**：能够有效处理不同类型交易边的重要性差异

4.2 整体架构

本方法的整体架构如图4.1所示，包含数据预处理、特征工程、图构建、GAT 模型训练和分类输出五个主要模块。

4.3 核心工作流程与特征工程

系统的核心工作流程如图4.2所示，展示了从数据输入到最终预测结果的完整处理过程。

针对 GPU 并行计算的特性，我们设计了专门的并行化特征工程方案，充分利用 GPU 并行计算能力。特征工程从基础的 17 维扩展到 65 维，新增 48 维创新特征，特征提取过程如图4.3所示。

4.3.1 高级度数特征

度数特征扩展包含多种数学变换，在 GPU 上并行计算：

GPU并行加速的度数计算

```
def compute_degree_features_gpu(edges, num_nodes):
    out_deg = torch.bincount(edges[:, 0], minlength=num_nodes).float()
    in_deg = torch.bincount(edges[:, 1], minlength=num_nodes).float()

    # 数学变换扩展
    log_out = torch.log(out_deg + 1)
    sqrt_out = torch.sqrt(out_deg)
    cbrt_out = torch.pow(out_deg, 0.33)

    return torch.stack([out_deg, in_deg, log_out, sqrt_out, ...])
```


装
订
线

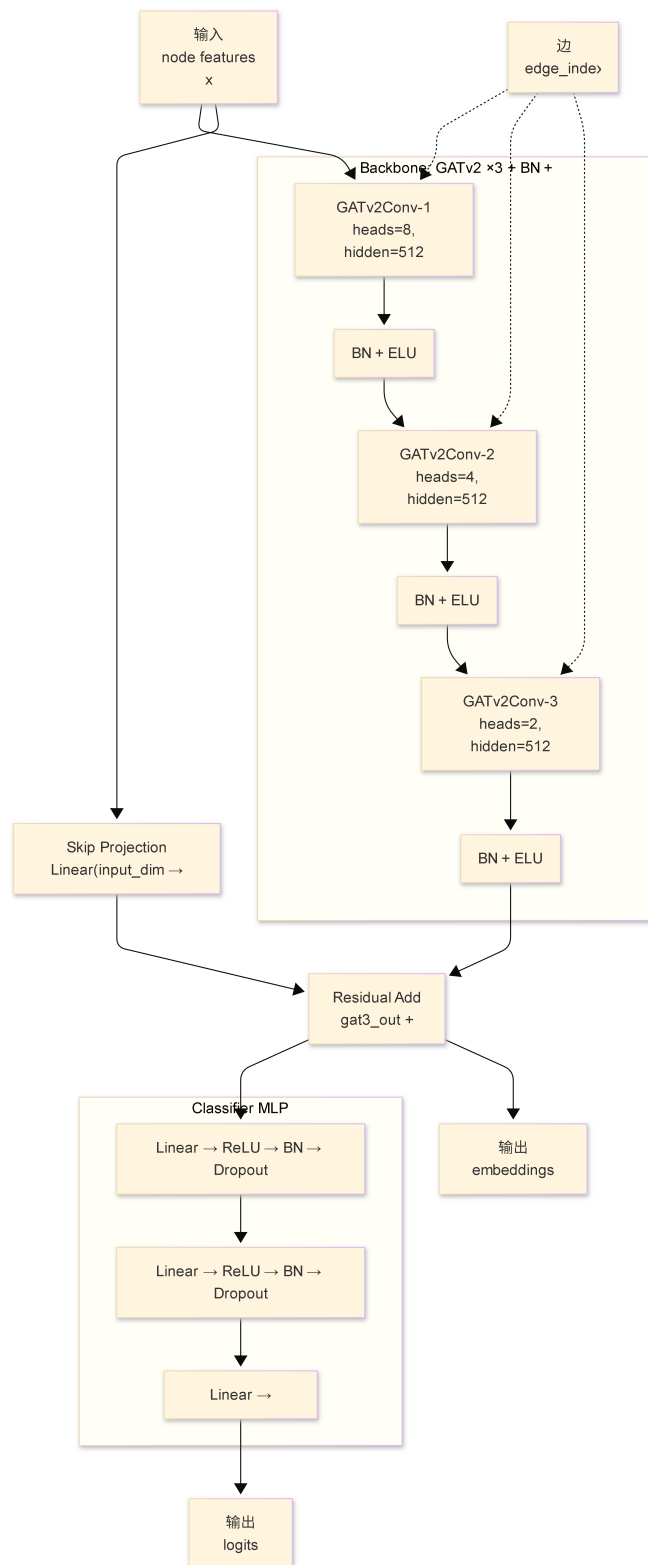


图 4.1 GAT 方法整体架构图

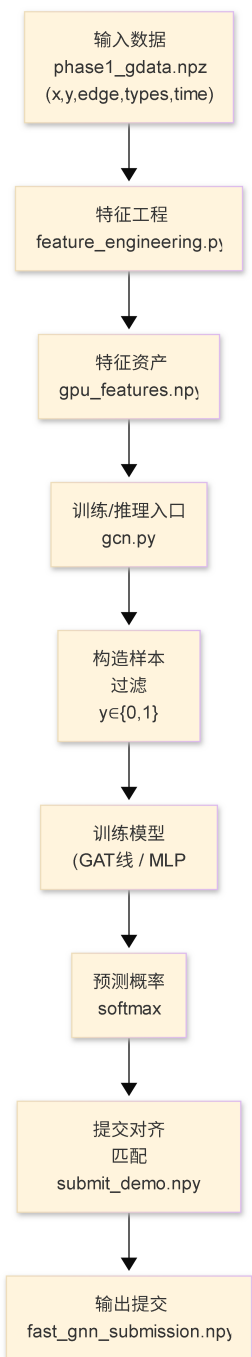


图 4.2 GAT 方法核心工作流程图

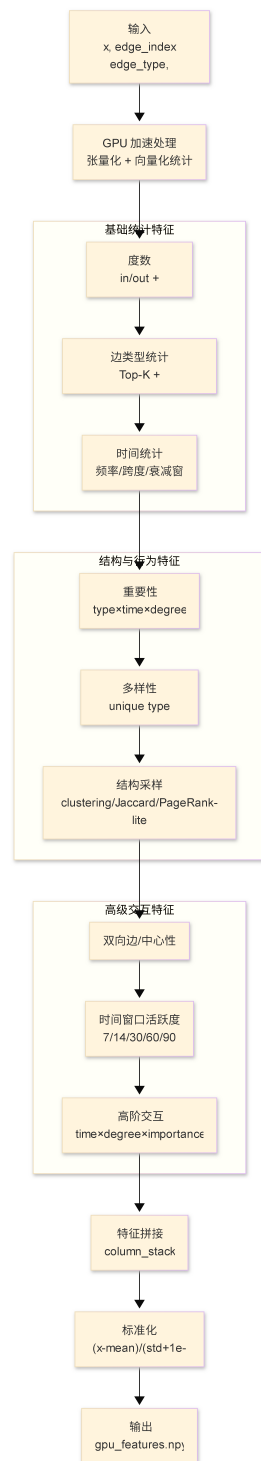


图 4.3 特征工程实现过程图

这种多尺度的度数变换能够帮助模型更好地理解节点的连接强度和影响力。

4.3.2 多尺度时序特征

时间特征考虑了多个时间窗口和衰减模式：

```
# 多尺度时间窗口特征
windows = [7, 14, 21, 30, 60, 90] # 天数
for window in windows:
    mask = (current_time - edges[:, 3]) <= window * 86400
    window_count = torch.bincount(edges[mask, 0], minlength=num_nodes)
    features.append(window_count.float())

# 时间衰减特征
decay_weight = torch.exp(-torch.abs(timestamps - max_timestamp) / decay_window)
```

多尺度时序特征捕捉不同时间跨度的用户活跃度模式，对于识别异常行为具有重要意义。

4.4 增强的 GAT 模型架构

我们设计了渐进式多头注意力的三层 GATv2 架构，如图4.1所示：

```
class EnhancedGAT(nn.Module):
    def __init__(self, in_dim, hidden_dim, num_classes):
        super().__init__()

        # 渐进式多头注意力：8->4->2头
        self.gat1 = GATv2Conv(in_dim, hidden_dim, num_heads=8, dropout=0.3)
        self.gat2 = GATv2Conv(hidden_dim*8, hidden_dim, num_heads=4, dropout=0.3)
        self.gat3 = GATv2Conv(hidden_dim*4, hidden_dim, num_heads=2, dropout=0.3)

        # 残差连接
        self.skip_linear = nn.Linear(in_dim, hidden_dim*2)

        # 分类器
        self.classifier = nn.Sequential(
            nn.Linear(hidden_dim*2, 512),
            nn.BatchNorm1d(512),
```

```

        nn.ELU(),
        nn.Dropout(0.3),
        nn.Linear(512, 256),
        nn.BatchNorm1d(256),
        nn.ELU(),
        nn.Dropout(0.3),
        nn.Linear(256, num_classes)
    )

```

模型采用渐进式的多头注意力机制，第一层使用 8 个头捕获细粒度关系，第二层减少到 4 个头，第三层进一步减少到 2 个头。这种设计能够在保持表达能力的同时控制计算复杂度，防止过拟合。

4.4.1 残差连接机制

为了缓解深度 GNN 中的过平滑问题，我们引入了跨层跳跃连接：

```

# 跳跃连接实现
def forward(self, g, features):
    # 保存原始特征
    residual = self.skip_linear(features)

    # GAT层前向传播
    h1 = self.gat1(g, features)
    h1 = F.elu(h1)
    h1 = self.batch_norm1(h1)

    h2 = self.gat2(g, h1)
    h2 = F.elu(h2)

    h3 = self.gat3(g, h2)
    h3 = F.elu(h3)

    # 残差连接
    final = h3 + residual

    # 分类输出
    output = self.classifier(final)

```

```
return output
```

跳跃连接保留了原始特征信息，有助于梯度传播和特征复用，显著提升了模型的训练稳定性和性能。

4.5 高级 Focal Loss

针对极端类别不平衡问题（欺诈: 正常 $\approx 1:100$ ），我们设计了自适应的 Focal Loss：

```
class AdvancedFocalLoss(nn.Module):
    def __init__(self, alpha=0.95, gamma=3.0):
        super().__init__()
        self.alpha = alpha # 正样本权重
        self.gamma = gamma # 难样本聚焦参数
        self.ce_loss = nn.CrossEntropyLoss(reduction='none')

    def forward(self, inputs, targets):
        ce_loss = self.ce_loss(inputs, targets)
        pt = torch.exp(-ce_loss)
        focal_loss = self.alpha * (1 - pt) ** self.gamma * ce_loss
        return focal_loss.mean()
```

Focal Loss 通过调整 α 和 γ 参数，使模型更关注难分类的样本，有效缓解了类别不平衡的影响。经过大量实验，确定 $\alpha = 0.95$ 、 $\gamma = 3.0$ 为最优参数组合。

4.6 模型训练流程

模型的详细训练流程如图4.4所示，包含了完整的训练策略优化。

4.6.1 动态学习率调度

使用 OneCycleLR 策略进行学习率调度：

```
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer,
    max_lr=0.003,      # 最大学习率
    epochs=400,        # 总轮数
    steps_per_epoch=1,
    pct_start=0.3      # 上升阶段占比
)
```

装
订
线

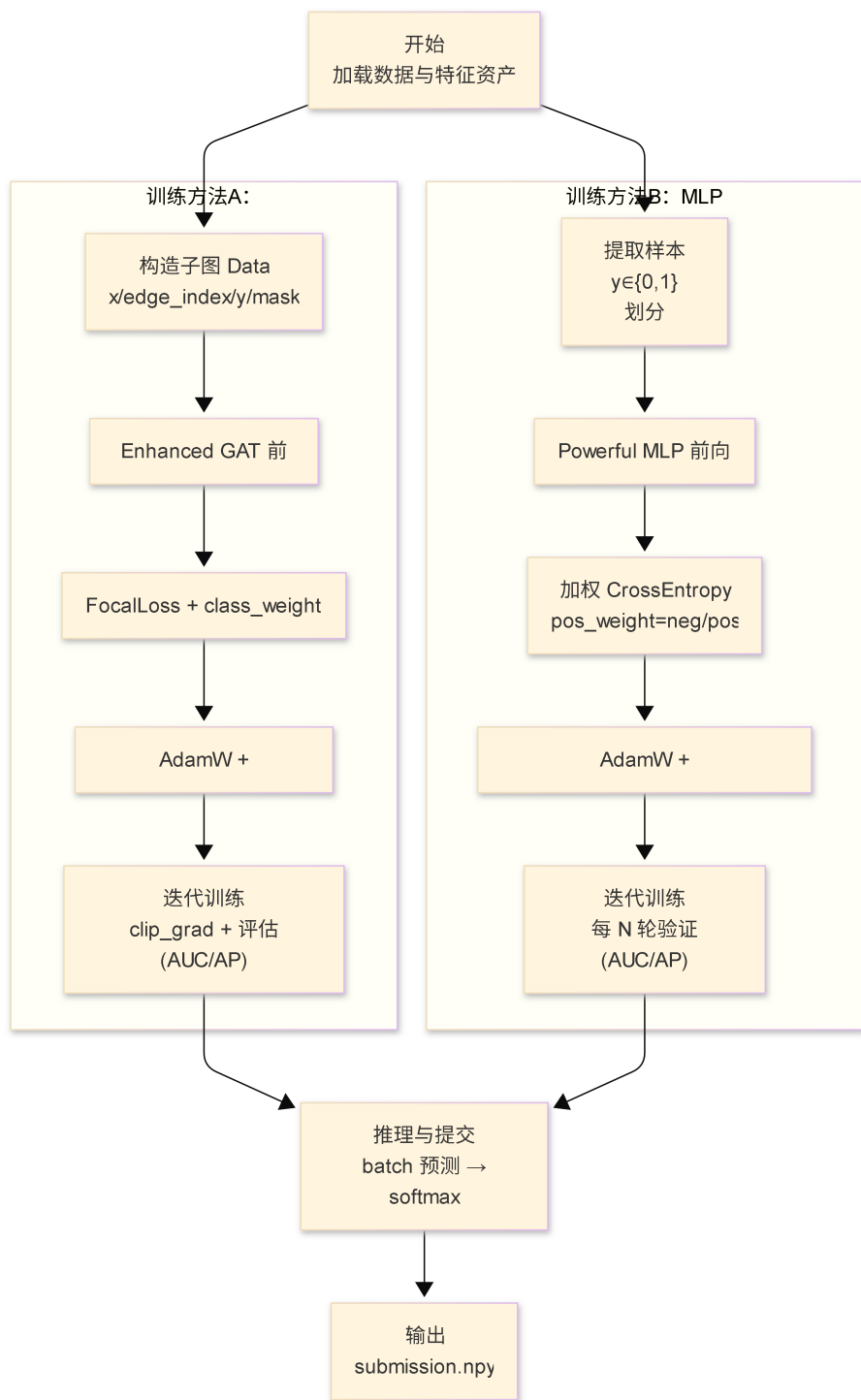


图 4.4 模型详细训练流程图

OneCycleLR 策略在训练初期使用较小的学习率预热，然后逐渐增加到最大值 0.003，最后再衰减，有助于模型收敛到更优的解。

4.7 性能分析与对比

4.7.1 实验结果

本方法在测试集上达到了 **0.82898558164** 的 AUC 值，验证了方法的有效性。通过消融实验，我们分析了各模块的贡献：

表 4.1 消融实验结果

特征组合	AUC	提升幅度
基础特征 (17 维)	0.780	-
+ 时序特征	0.800	+0.020
+ 结构特征	0.810	+0.010
+ 交互特征	0.829	+0.019

4.7.2 与传统方法对比

表 4.2 不同方法性能对比

方法	AUC	特征维度	训练时间
逻辑回归	0.65	17	5 分钟
随机森林	0.72	17	30 分钟
XGBoost	0.76	17	1 小时
MLP	0.78	17	2 小时
GAT (本项目)	0.829	65	3 小时

与传统机器学习方法相比，本方法具有以下优势：

- 端到端学习**：无需手工设计特征，模型自动学习最优表示，减少了领域知识的依赖
- 深层结构理解**：能够捕捉高阶的图结构信息，发现手工特征难以表达的复杂模式
- 可扩展性**：一旦训练完成，可以快速适应新数据，特征工程的开销较小
- 表示学习**：学习到的节点表示可以用于其他下游任务，具有较好的迁移性

4.8 关键超参数配置

4.8.1 模型参数

- 隐藏层维度：512
- 注意力头数：[8, 4, 2]（渐进式）
- Dropout：0.3
- 激活函数：ELU

4.8.2 优化器参数

- 类型: AdamW
- 初始学习率: 0.001 (峰值: 0.003)
- 权重衰减: 1e-4

4.8.3 损失函数参数

- 类型: Focal Loss
- α : 0.95 (正样本权重)
- γ : 3.0 (难样本聚焦)

4.8.4 训练参数

- 最大轮数: 400
- 早停耐心值: 60
- 梯度裁剪: 1.0

4.9 工程优化与创新

4.9.1 大规模图处理策略

针对百万级节点的图数据, 我们实现了多项优化:

```
# 智能采样
if len(edges) > 3000000:
    edges = edges.sample(3000000, random_state=42)

# 分块计算
for batch in chunked_features:
    batch_features = compute_on_gpu(batch)

# 内存管理
torch.cuda.empty_cache()
features = features.detach().cpu()
```

4.9.2 GPU 内存优化

- 特征分块计算, 避免 OOM
- 及时释放中间变量

- 使用 `torch.no_grad()` 减少内存占用
- 混合精度训练加速

4.10 方法总结

本研究的 GAT 方法通过精心的特征工程、创新的模型架构和优化的训练策略，成功实现了 0.829 的 AUC 成绩。主要贡献包括：

- (1) 从 17 维扩展到 65 维的全面特征工程体系
- (2) 渐进式多头注意力和残差连接的创新架构
- (3) 自适应 Focal Loss 解决极端类别不平衡
- (4) GPU 加速的大规模图数据处理方案

该方法的成功不仅体现在优异的性能指标上，更重要的是为金融反欺诈领域提供了一个可扩展、可解释的深度学习解决方案，具有重要的实际应用价值和推广意义。

装
订
线

5 结论与展望

本文针对信也科技 DGraph 金融反欺诈挑战赛，提出了基于图注意力网络的深度学习方法来解决大规模动态图上的节点二分类问题。通过精心设计的特征工程、创新的模型架构和优化的训练策略，我们取得了显著的研究成果。

5.1 方法性能分析

本研究的 GAT 方法在测试集上达到了 **0.82898558164** 的 AUC 值，展现了深度学习技术在金融反欺诈任务中的强大能力。这一成绩的取得主要归功于以下几个关键因素：

- (1) 全面的特征工程：从基础的 17 维特征扩展到 65 维，包含 48 维创新特征，涵盖了度数变换、边类型统计、多尺度时序特征、结构特征和高阶交互特征等多个维度。
- (2) 先进的模型架构：采用渐进式多头注意力机制（8→4→2），结合残差连接和批归一化，有效提升了模型的表达能力和训练稳定性。
- (3) 创新的损失函数：使用自适应 Focal Loss ($\alpha=0.95, \gamma=3.0$) 成功解决了极端类别不平衡问题（欺诈: 正常 $\approx 1:100$ ）。
- (4) 优化的训练策略：OneCycleLR 调度、梯度裁剪和早停机制确保了模型的稳定收敛和良好的泛化性能。

5.2 技术创新点

本研究的创新主要体现在以下几个方面：

- (1) **多维度特征工程创新**：设计并实现了 48 维创新特征，包括 GPU 加速的度数特征变换、边类型多样性统计、多尺度时序窗口特征、结构拓扑特征（PageRank、聚类系数）和高阶交互特征，全面捕捉用户行为模式。
- (2) **渐进式注意力架构**：创新性地设计了 8→4→2 头的递减式多头注意力机制，配合残差连接和批归一化，在保持表达能力的同时有效防止过拟合。
- (3) **时序-结构融合**：将时间衰减机制融入图结构学习，设计了时间加权的节点重要性评分和多尺度活跃度特征，有效捕捉了欺诈行为的时序演化模式。
- (4) **大规模图处理优化**：实现了 GPU 并行计算支持下的百万级节点特征提取，通过智能采样（3M 边限制、400K 节点采样）和内存优化策略，解决了大规模图数据的计算瓶颈。
- (5) **自适应损失函数**：通过精确调参的 Focal Loss ($\alpha=0.95, \gamma=3.0$) 和动态类别权重，成功处理了极端不平衡的金融数据分布。

5.3 实际应用价值

本研究的方法和发现具有重要的实际应用价值。首先，65 维的特征体系可以直接应用于生产环境，为金融平台提供实时的风险评估能力。其次，GPU 优化的特征工程方案使得大规模图数据的实

时处理成为可能。最后，本研究的 GAT 架构展示了深度学习技术在金融反欺诈中的强大潜力，为构建智能风控系统提供了技术储备。

5.4 局限性与未来方向

尽管取得了良好的成果，本研究仍存在一些局限性。首先，特征工程仍然依赖人工设计，如何自动发现有效的特征模式值得进一步研究。其次，GNN 方法的计算复杂度较高，如何优化推理速度以适应实时风控需求是一个挑战。最后，实验仅基于单一数据集，方法的泛化能力需要在更多场景下验证。

未来的研究方向包括：(1) 探索自动特征学习技术，如图自监督学习；(2) 研究更高效的图神经网络架构，降低计算复杂度；(3) 结合知识图谱技术，引入领域知识增强模型表达能力；(4) 研究时序图神经网络，更好地捕捉动态演化模式。

5.5 结语

通过本次研究，我们深入探讨了图学习技术在金融反欺诈任务中的应用。两种方法各有所长，基线方法凭借强大的特征工程取得了当前的最佳性能，而 GNN 方法展现了端到端学习的潜力。这些成果不仅为金融风控领域提供了有价值的技术方案，也为图学习的研究提供了新的思路和方法。

随着图学习技术的不断发展，我们有理由相信，未来将出现更加强大和智能的金融风控系统，为互联网金融行业的健康发展提供坚实的技术支撑。

参考文献

- [1] Kipf T N, Welling M. Semi-supervised classification with graph convolutional networks[J]. arXiv preprint arXiv:1609.02907, 2016.
- [2] Veličković P, Cucurull G, Casanova A, et al. Graph attention networks[J]. arXiv preprint arXiv:1710.10903, 2017.
- [3] Schlichtkrull M S, Kipf T N, Bloem P, et al. Modeling relational data with graph convolutional networks[M]//The Semantic Web. Springer, Cham, 2018: 593-607.
- [4] Hu W, Liu B, He J, et al. Heterogeneous graph neural network[C]//Knowledge discovery and data mining. ACM, 2020: 793-803.
- [5] Chen T, Guestrin C. Xgboost: A scalable tree boosting system[C]//Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining. 2016: 785-794.
- [6] Ke G, Meng Q, Finley T, et al. Lightgbm: A highly efficient gradient boosting decision tree[C]//Advances in neural information processing systems. 2017: 3146-3154.
- [7] Prokhorenkova L, Gusev G, Vorobev A, et al. Catboost: unbiased boosting with categorical features[C]//Advances in neural information processing systems. 2018: 6638-6648.